TEMPLATE FROM @DOMBUILDS.AI

The Token Leak Kit

All 3 GitHub links, plus copy-paste setup and safety checks to make your Claude Code limit last longer.

> No repo changes your plan, so these three will not literally turn Pro into Max, but they can cut the junk Claude reads and writes. Install them biggest leak first, because anything that enters the context is carried through later turns.

Use it in 5 minutes

1. Copy the 3 links and run the baseline check in a Claude Code session.
2. Set up RTK now, then add Serena if your codebase is large.
3. Try Caveman last and keep only what helps. If a command fails, follow the repo's latest README.

THE 3 LINKS

01 **RTK: trims terminal output**

Start here if Claude runs tests, builds, logs, or git commands often. It filters noisy command output before Claude has to read it.

```
https://github.com/rtk-ai/rtk
```

02 **Serena: finds code by symbol**

Best for larger codebases. It uses code intelligence so Claude can locate one function and its references without opening entire files.

```
https://github.com/oraios/serena
```

03 **Caveman: shortens replies**

Add this last. It cuts wording from Claude's replies, but your files, tool results, and terminal output can still be the bigger leak.

```
https://github.com/JuliusBrussee/caveman
```

BEFORE YOU INSTALL

04 Capture your baseline

Run this after a typical task and screenshot the result. Repeat after each install, using similar work, so you can see what actually changed.

```
/context
```

05 Review what will run

These tools can add hooks, servers, or instructions to your setup. Run this check for each repo, swapping the link at the end, before giving it access to your machine.

```
Review the GitHub repo linked at the end of this message before I install it. Summarize what its installer changes, which files it edits, what commands it runs, and whether it sends data anywhere. Do not install anything yet. Repo: https://github.com/rtk-ai/rtk
```

SET UP IN THIS ORDER

06 Install RTK

This uses Homebrew and sets up the global Claude Code hook. Restart Claude Code afterward so the hook loads. On Linux or Windows, use the README instructions.

```
brew install rtk
rtk init -g
```

07 Confirm it is the right RTK

Another package also uses the name rtk. If this command is not recognized, you may have installed the wrong one. Once working, it shows your RTK savings.

```
rtk gain
```

08 Install Serena through Claude

Serena's recommended command can change between releases, so have Claude read the current instructions. Approving the command first keeps you in control of what runs.

```
Read https://github.com/oraios/serena and find the current setup for Claude Code. Show me the exact install command and any prerequisites for this project. Wait for my approval before running anything.
```

09 Tell Claude to use Serena

Installing an MCP server does not mean Claude will reach for it at the right moment. Paste this at the start of code navigation tasks, then describe your task after it.

```
For the task below, use Serena's symbol tools, such as find_symbol and find_referencing_symbols, before opening files. Read a whole file only when symbols do not give you enough context.
```

10 Install Caveman

Run these in your terminal, then restart Claude Code. Test it on a normal coding task before deciding whether the shorter style suits you.

```
claude plugin marketplace add JuliusBrussee/caveman
claude plugin install caveman@caveman
```

11 Start with the lite mode

Very terse replies are not worth it if you have to ask Claude to explain them. Start light and increase only if the answers stay clear.

```
/caveman lite
```

AVOID THE TRAPS

12 Know RTK's blind spot

RTK's hook works on shell commands. Claude's built-in file reading and search tools bypass it, so a good RTK result does not mean every leak is fixed.

```
When you need to inspect code, avoid reading whole files by default. Search for the exact symbol or text first, then read only the relevant section.
```

13 Get raw output when debugging

Filtered output can occasionally hide the clue you need. When a failure makes no sense, run that one command unfiltered, then return to normal. Example shown with npm test, so use your own command.

```
rtk proxy npm test
```

14 Check that Serena earns its place

MCP tools take up context too, and a small project may not benefit. Compare similar tasks with Serena on and off, then keep whichever uses less.

```
Look at this session and tell me whether Serena reduced whole-file reads.
List which files you opened fully and whether symbol lookup could have
replaced each read.
```

15 Protect anything that must be precise

Short chat is fine, but vague warnings or commit messages can cost more later. Add this rule to your CLAUDE.md to keep important writing clear.

```
Terse style applies only to chat replies. Use normal, precise language for
code, commit messages, documentation, destructive command warnings, and
security issues.
```

16 Use the free fix too

No repo helps if one conversation drags old work into a new task. Clear the session whenever you switch to something unrelated.

```
/clear
```

ILLUSTRATIVE TEST RUN WITH RTK

Before: Example: Claude runs the test suite and receives every passing test, progress line, and warning, even though only one test failed.

After: Example: Claude receives the failing test, its file, and the error, which is what it needs to start the fix.

Next

I share more tools and workflows I test while building apps with AI on @dombuilds.ai, including what turned out not to be worth keeping.

Follow @dombuilds.ai for the next one.